一文了解 Linux 内核的内存管理 - 知乎

编辑于 2021-08-08 · 阅读量 1.6万+ 内核是操作系统的核心，而内存管理是内核最复杂的子系统之一。本文将从宏观视角出发，带你快速建立对 Linux 内存管理的整体认知，为后续深入学习打下基础。

内核空间与用户空间

Linux 采用经典的 3:1 内存布局，将 4GB 虚拟地址空间分为两部分：

- 用户空间：0x00000000 - 0xBFFFFFFF（3GB）
- 内核空间：0xC0000000 - 0xFFFFFFFF（1GB）

内核空间内部又细分为多个区域：

- 直接映射区（Direct Mapping）：内核代码、数据、页表、slab 分配器等，物理地址与虚拟地址一一对应
- vmalloc 区：用于大内存分配，虚拟连续但物理不连续
- 高端内存区（Highmem）：仅 32 位系统需要，用于访问超过 1GB 的物理内存

物理内存组织

Linux 将物理内存按 NUMA 节点（Node）组织，每个 Node 内部按 Zone 进一步分区：

- ZONE_DMA：0 - 16MB，用于老设备 DMA
- ZONE_DMA32：0 - 4GB，用于 32 位 DMA 设备
- ZONE_NORMAL：内核可直接寻址的内存
- ZONE_HIGHMEM：仅 32 位系统，内核需临时映射才能访问

每个 Zone 内部以 4KB 页（Page）为最小单位管理，内核通过 struct page 结构体记录每个物理页的状态。

伙伴系统（Buddy System）

伙伴系统是内核分配物理内存的核心算法，解决外部碎片问题。

- 核心思想：将空闲内存按 2^n 个页的大小分成多个链表，分配时拆分大块，释放时合并相邻空闲块
- 分配 4KB：从 order=0 链表取一个页
- 分配 8KB：从 order=1 链表取一个页，若无则拆 order=2 的大块
- 释放：检查伙伴是否空闲，若是则合并成 order+1 的大块

slab 分配器

伙伴系统以页为单位分配，对内核频繁创建的小对象（如 task_struct、inode、dentry）效率太低。slab 分配器在伙伴系统之上，为每种内核对象维护一个缓存池，实现对象的复用与快速分配。

- 每种对象一个 slab 缓存，预分配若干页，切分成固定大小的对象
- 分配对象：从空闲链表取一个对象
- 释放对象：归还到空闲链表，不立即释放内存
- 内核模块：slabinfo 命令查看各 slab 缓存的使用情况

虚拟内存与 MMU

每个进程拥有独立的虚拟地址空间，通过 MMU 硬件实现虚拟地址到物理地址的翻译。

- 页表：多级树结构，叶节点记录虚拟页对应的物理页帧
- 缺页异常：访问未映射的虚拟地址，内核分配物理页并建立映射
- 写时复制（COW）：fork 后父子进程共享只读页，写入时才复制

内存回收与 OOM

当内存不足时，内核通过两种方式回收内存：

- 后台回收：kswapd 内核线程异步扫描，优先回收缓存页、脏页写回、换出匿名页
- 直接回收：分配内存时发现不足，当前进程同步执行回收，会阻塞分配

若回收后仍不足，内核触发 OOM Killer，按进程内存占用与优先级打分，杀掉最合适的进程以释放内存。

内存分配接口

- kmalloc：内核通用小内存分配，底层走 slab
- vmalloc：大内存分配，虚拟连续但物理不连续
- alloc_page：直接分配物理页
- kmem_cache_alloc：从指定 slab 缓存分配对象

调试工具

- /proc/meminfo：全局内存统计
- /proc/slabinfo：slab 缓存统计
- /proc/vmstat：内存子系统计数器
- /proc/zoneinfo：各 Zone 水位与状态
- /proc/buddyinfo：伙伴系统各 order 空闲块数量
- slabtop：实时查看 slab 缓存占用
- free -m：快速查看内存概况

小结

掌握内存管理的核心脉络：内核空间与用户空间的隔离、物理内存按 Node/Zone 组织、伙伴系统管理页级内存、slab 管理小对象、MMU 实现虚拟内存、回收与 OOM 兜底。这是理解 Linux 性能调优、内存泄漏排查、容器资源隔离等高级话题的基础。

参考资料

- 《深入理解 Linux 内核》第 3 章
- 《Linux 内核设计与实现》第 10 章
- kernel.org 官方文档

关注我们获取更多内容
关注微信公众号，回复 “Linux” 即可获取 Linux 内核、网络、存储、虚拟化、容器、性能调优等方向的高质量文章推送。
关注微信公众号，回复 “Linux” 即可获取 Linux 内核、网络、存储、虚拟化、容器、性能调优等方向的高质量文章推送。
关注微信公众号，回复 “Linux” 即可获取 Linux 内核、网络、存储、虚拟化、容器、性能调优等方向的高质量文章推送。